v0.30.0 | Autonomous Execution Agent | SOLO 35-day build

GEODE

`while(tool_use) { observe → learn → act }`

> **확률적 시스템(LLM)은 제어 없이 발산합니다.**
>
> 발산을 루프와 가치로 수렴시키는 하네스를 만듭니다.

185	3,219+	38	0	1,118
Python Modules	Tests Passing	Releases	Regressions	Commits

35 days (Feb 21 — Mar 27) · 52 tools + 41 MCP · 40 Hook events · 17 CANNOT rules

Harness Engineering — 4-Axis Framework

확률적 LLM을 신뢰 가능한 시스템으로 수렴시키는 4개 제어 축. 루프가 데이터를 생산하고, 관측이 포착하고, 검증이 확인하고, 컨텍스트가 다음 반복을 제어한다.

컨텍스트 제어
Context Control

정보 주입 설계

PromptAssembler 6-Phase: template → override → skill → memory → bootstrap 순서로 동적 조립. LLM이 보는 것이 곧 결과.

`SHA-256 prompt caching`

7개 노드 동일 프롬프트 공유 → 입력 비용 **90%** 절감. 200 turns sliding window.

실행 루프
Execution Loop

오케스트레이션

AgenticLoop while(tool_use), 최대 **50** 라운드. 자연 종료·수렴 감지(동일 에러 4회)·강제 텍스트 등 5가지 경로로 종료.

`54 tools × 50 rounds` 자율 실행

5-Tier 안전 등급. TIER 0-1 병렬, TIER 2 비용 일괄 승인, TIER 3-4 HITL 순차. 3-Provider Failover.

검증
Verification

비결정론 → 신뢰

Swiss Cheese 3-Layer → GEODE 5-Layer로 진화: 69.4 → **99.8** 정확도. 단일 LLM 평가의 Confirmation Bias 제거.

`GEODE 5-Layer + Cross-LLM`

G1-G4 Guardrails → BiasBuster(6종) → Claude×GPT 교차검증(α≥0.67) → Confidence Gate(0.7).

관측
Observability

작동 증명

40-Event Hook Observer가 파이프라인 전 생명주기를 포착. LEARNING.md에 자동 축적. 루프가 작동함을 증명해야 신뢰 가능.

`LangSmith / OTEL / CUSUM`

TokenTracker: 18개 모델 가격 레지스트리. 컨텍스트 80% 경고, 95% 위험 시 adaptive pruning 발동.

`Context` → `Loop` produces data → `Observability` captures → `Verification` validates → `Context`

4개 축이 순환하며 LLM의 발산을 수렴으로 제어한다

Triple Loop: META, DEV, RUNTIME

while(tool_use) — 세 루프의 공유 기본 단위. RUNTIME이 DEV를 먹이고, DEV가 META를 먹인다.

META
Scaffold Evolution
weeks

Observe
CI 결과 · DRIFT_DETECTED · 세션 회고

→

Learn
패턴 추출 · anti-pattern 식별

→

Update Scaffold
CLAUDE.md 규칙 추가 · 스킬 21개 · CI 래칫 ≥2900

controls

PIPELINE_END · TURN_COMPLETE → LEARNING.md 업데이트 → CLAUDE.md CANNOT 규칙 추가 · 스킬 생성

DEV
Claude Code per-PR
hours

Plan
Socratic 5Q · GAP Audit

→

Implement
feature branch · ruff+mypy+pytest

→

Verify
4-persona 리뷰 · CI 5/5

→

Merge
PR→main · progress.md 업데이트

produces

SESSION_START → GEODE.md + MEMORY.md + LEARNING.md 로드 · VERIFICATION_FAIL → 새 DEV PR 트리거

RUNTIME
GEODE while(tool_use)
seconds

Query
사용자 입력 · SESSION_START

→

Tool Select
52 native + 41 MCP · 4-경로 디스패치

→

Execute
PolicyChain 6-layer · conf ≥0.7

→

Observe
40 Hook 이벤트 · 메모리 저장

spawns

STRUCTURAL ISOMORPHISM

Harness Builds Harness

SCAFFOLD(빌더)와 GEODE(결과물)는 동일한 아키텍처를 공유한다. 각 컴포넌트는 1:1 구조적 거울을 갖는다.

SCAFFOLD

Claude Code — GEODE를 빌드

CLAUDE.md — 425줄, 17 CANNOT 규칙
5개 도메인: Git · Planning · Quality · Docs · PR · 개발자 에이전트 SOT

21 Scaffold Skills (.claude/skills/)
geode-gitflow · kent-beck-review · karpathy-patterns · verification-team + 17개

CI 5-Job Ratchet
Lint(ruff) · Type(mypy) · Test(pytest ≥2900) · Security(bandit) · Gate

CANNOT/CAN (17 hard rules + 5 freedoms)
자연어 제약 — LLM이 읽고 내면화. CI에서 강제 가능한 hard rule만 포함.

4-Persona Review
Kent Beck · Karpathy · Steinberger · Cherny — 다중 관점 PR 리뷰 시뮬레이션

Hook
40
events

`while (tool_use)`

Shared Primitive

GEODE

Autonomous agent runtime

GEODE.md — 71줄, 4 CANNOT 규칙
SESSION_START 시 core/memory/organization.py를 통해 로드 · 런타임 에이전트 정체성

Runtime Skills (.geode/skills/)
4개 분석가 템플릿: discovery · game-mechanics · growth-potential · player-experience

5-Layer Verification (core/verification/)
G1-G4 Guardrails · BiasBuster · Cross-LLM(α≥0.67) · Confidence Gate 0.7 · Rights Risk

PolicyChain 6-Layer (core/tools/policy.py)
Profile · Org · Mode · Agent(STANDARD/WRITE/DANGEROUS) · Node-scope · Sub-agent

BiasBuster — 6 유형 편향 제거
confirmation · recency · anchoring · position · verbosity · self-enhancement

하네스를 빌드하는 하네스는 자신의 구조를 공유한다.

CANNOT / CAN Framework

제약이 품질을 담보한다. 486 PRs, 0 hotfix bypasses, 0 regressions.

strictness → 0 regressions

CANNOT — 17 Hard Rules (CLAUDE.md)

Git / 격리 (6)

- worktree 없이 코드 작업 금지
- main/develop 직접 push 금지
- 타 세션 worktree 삭제 금지
- 원격 미동기화 상태 브랜치 생성 금지

Docs / PR (6)

- 코드 커밋에서 CHANGELOG 누락 금지
- 버전 4곳 불일치 금지
- CI 가드레일 미통과 PR 머지 금지
- Why 근거 없이 PR 금지

Quality (4)

- lint/type/test 실패 상태 커밋 금지
- 수치에 자리표시자(XXXX) 금지
- live 테스트(-m live) 무단 실행 금지

설계 원칙

CANNOT에 없는 것은 자유롭게 할 수 있다. 17개 경계가 에이전트의 과잉 구현 충동을 구조적으로 제거한다.

CAN — 5 Unconstrained Freedoms

- 버그/문서 수정 — Plan 없이 바로 구현
- 플랜 외 개선 발견 — 현재 완료 후 다음 이터
- 테스트 선별 실행 — 최종만 전체
- 커밋 메시지 언어 — 한글/영어 자유
- 도구 선택 — 더 빠른 도구 선택 가능

STAR Result

486	0	0
PRs merged	hotfix bypasses	regressions

모든 PR은 5-job CI를 통과해야만 merge 가능

8-Step Engineering Loop

486 PRs, 0 regressions — GAP Audit + Socratic Gate가 과잉 구현을 소스에서 차단한다.

0 Worktree → 1 GAP Audit → 2 Socratic Gate → 3 Implement → 4 E2E Verify → 5 Docs-Sync → 6 PR + CI Gate → 7 Board

Socratic 5-Question Gate

- **Q1** 코드에 이미 있는가? grep/Explore 실측 — 있으면 제거.
- **Q2** 이걸 안 하면 무엇이 깨지는가? 실제 장애 시나리오 — 답 없으면 제거.
- **Q3** 효과를 어떻게 측정하는가? 측정 불가 시 보류.
- **Q4** 가장 단순한 구현은? 최소 변경만 채택.
- **Q5** 프론티어 3종 이상에서 동일 패턴인가? 1종만이면 재검증.

CI 5-Job Ratchet

- **lint** — ruff check + import ratchet
- **typecheck** — mypy + prompt integrity
- **test** — pytest count >= **2,900** (현재 3,249+)
- **security** — bandit -r core/
- **gate** — 4/4 all pass to merge

Recursive Recovery

Step 4 E2E fail → Step 3 (Implement)

Step 6 CI fail → Step 3 (Implement)

Frontier Mining + 4-Persona Review

3종 이상 프론티어에서 수렴한 패턴만 채택하고, 4인 페르소나가 단일 리뷰어가 놓치는 것을 병렬로 감사한다.

autoresearch
Karpathy Autonomous Loop

P1→CANNOT 규칙. P4→CI ratchet. P6→컨텍스트 압축. P10→최소 구현.

OpenClaw
Steinberger / PSPDFKit

Gateway Session Key. Lane Queue 동시성. Atomic write for .env 보안.

Claude Code
Cherny / Anthropic

while(tool_use) AgenticLoop. HITL 3-tier SAFE/WRITE/DANGEROUS. Sliding-window context.

Codex CLI
OpenAI

Worktree 격리 + .owner 소유권. Bash resource limits. Secret redaction.

Kent Beck
XP creator · TDD inventor

가장 단순한 구현인가? 불필요한 추상화가 있는가? Error paths 테스트됐는가?

Karpathy
Tesla AI Director · autoresearch

Token guard? Ratchet? 이 기능이 컨텍스트 윈도우를 얼마나 소비하는가?

Steinberger
PSPDFKit founder · OpenClaw

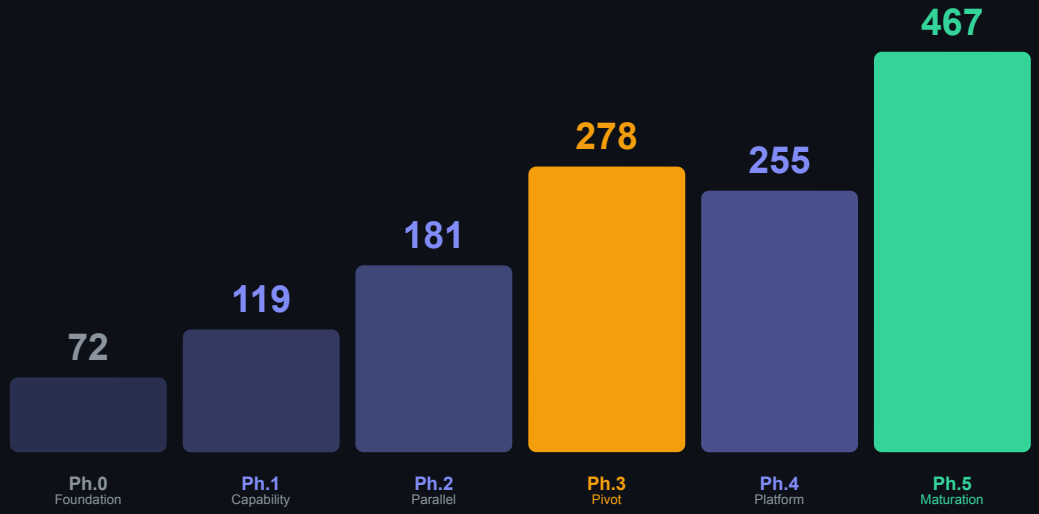
Atomic write (tmp+rename)? 크래시 시 상태 파일이 깨지지 않는가?

Cherny
Claude Code Lead · ex-Meta

Tool definitions clear? HITL classification correct? while(tool_use) 루프가 올바른가?

Compound Iteration Velocity

35일, 1,118 커밋, 35 releases — 엔지니어링 시스템이 완성될수록 속도는 복리로 증가했다.



1,118 commits · 486 merged PRs · 35 releases · 0 hotfix bypasses · 0 regressions

Phase 3: Identity Pivot — 2일, 6 releases

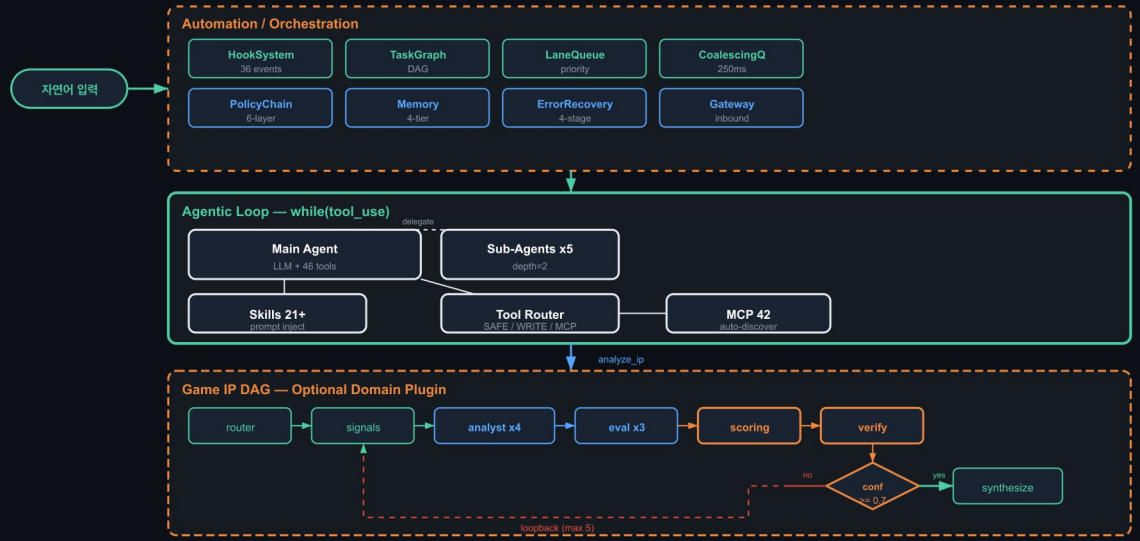
v0.13.0: "Undervalued IP" → 범용 에이전트. DomainPort Protocol로 도메인 분리.

v0.15.0: Token Guard 철폐 — 하드 캡 대신 압축. max_turns 20→200.

4/day → 67/day = **16.7x**

6-Layer Architecture

monolithic runtime.py 1,476줄에서 185개 모듈, 6개 레이어로 — 새 도메인은 DomainPort Protocol 12 메서드만 구현하면 된다.



Layer Module Counts

L0 CLI + Agent	26 + 8
L1 Infra + Gateway	16 + 13
L2 Memory	14
L3 Orchestration	14
L4 Tools + MCP	13 + 18
L5 Domain Plugins	14
TOTAL	**185**

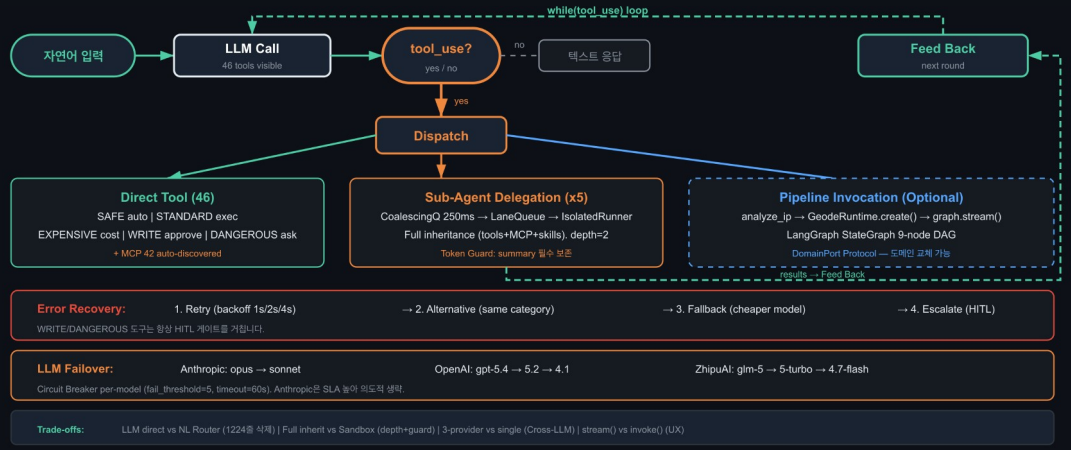
Hook System — Cross-cutting

core/hooks/ (7 모듈)는 L0–L5 전 레이어에서 import 가능한 횡단 관심사. 종속 사이클 해소를 위해 분리 추출.

45 events · 17 handlers

AgenticLoop — while(tool_use)

LLM이 tool_use를 반환하는 한 최대 50 라운드까지 루프를 계속하고, 5가지 경로로 종료한다.



5가지 종료 경로

- **natural** — LLM이 end_turn 반환 — 자연 완료.
- **forced** — 잔여 2라운드에서 텍스트 강제 추출.
- **convergence** — 동일 에러 4회 반복 → 조기 종료.
- **llm_error** — 연속 2회 실패 → Cross-Provider Fallback.
- **cancelled** — Ctrl+C 인터럽트 또는 billing 에러.

컨텍스트 관리 3단계

- **200 turns** — Sliding window — 최초 메시지 보존, 중간 제거.
- **80%** — 컨텍스트 80% → compact 압축 발동.
- **95%** — 95% → emergency prune 자동 발동.

Tool Dispatch: 단일 진입점 → 4경로 분기

ToolExecutor가 LLM tool_use를 받아 도구 유형과 안전 등급으로 라우팅합니다

LLM이 tool_use 블록을 반환하면 **ToolCallProcessor**가 도구 수를 먼저 판단합니다. 단일 도구는 바로 실행, 2개 이상이면 **5-Tier 안전 등급**으로 분류 후 Tier에 따라 병렬 또는 순차 실행합니다.

DISPATCH DECISION TREE

ToolExecutor.execute()
단일 진입점 — 모든 도구 디스패치

- **Bash Path** — run_bash? — 3-layer safety
- **Native Tool** — registry 핸들러? — 52 native tools
- **MCP Path** — MCP 서버? — 44 catalog
- **LangGraph DAG** — analyze_ip? — 13-task graph

↓ 2개 이상 tool_use 시 — 5-Tier 안전 등급 분류

- **TIER 0** — SAFE — gather 병렬
- **TIER 1** — MCP Auto — 서버당 1회
- **TIER 2** — EXPENSIVE — 일괄 승인
- **TIER 3** — WRITE — 개별 HITL
- **TIER 4** — DANGEROUS — 차단 + HITL

Error Recovery Chain

연속 2회 실패 시 자동 복구 체인 실행

1 Retry	동일 도구 재시도 (1s backoff)
2 Alt	동일 카테고리 대체 도구
3 Fallback	더 저렴한 도구로 강등
4 Escalate	사용자에게 에스컬레이션

Trade-off

Tier 0-1은 **asyncio.gather**로 완전 병렬 실행하여 응답 속도를 극대화하고, Tier 3-4는 순차+개별승인으로 안전성을 보장합니다.

`MAX_CONSECUTIVE_FAILURES = 2 · max_recovery = 3`

Hook System: 하드코딩 콜백 → 45-Event Bus

재귀 개선·자기 증강을 위한 실시간 추적 — Claude Code·Devin·OpenHands 리서치 후 개편

문제: DAG 노드 상태·검증 결과·드리프트를 실시간 추적하려면 하드코딩 콜백으로는 확장이 불가능했습니다. 프론티어 하네스를 리서치하고 **8개 카테고리 45개 이벤트**로 개편했습니다.

리플 패턴: 하나의 이벤트가 L1(관측)과 L2(반응)를 동시에 지나가는 것이 리플 패턴입니다.

6 CORE EVENT CATEGORIES

Pipeline Lifecycle
3 events — START · END · ERROR

Node Execution
4 events — ENTER · EXIT · ERROR

L4.5 Automation
5 events — DRIFT · OUTCOME · TRIGGER

Context Overflow
3 events — 80% · 95% · ACTION

SubAgent
3 events — STARTED · DONE · FAILED

Tool HITL
3 events — REQUESTED · GRANTED · DENIED

HOOK MATURITY MODEL

L4 Autonomy — 백로그
승인 이력 → 자동 승인 룰 학습

L3 Decide — 현재 프론티어
✓ CONTEXT_OVERFLOW_ACTION 전략 위임

L2 React — 자동 반응
✓ DRIFT → 스냅샷 · PIPELINE_END → MEMORY.md

L1 Observe — 기록만
✓ RunLog: ALL 45 events → runs.jsonl

StuckDetector

NODE_ENTER 후 **120초** 내 NODE_EXIT 없으면 파이프라인 강제 중단. Priority 기반 핸들러 순서 보장 (P30→P50→P85).

Sub-Agent: 이중 주입 제거 + 동기/비동기 분리

tool_result와 announce가 같은 정보를 2회 주입하는 문제. OpenClaw 패턴 참조, 2파일 4줄로 해결

PROBLEM: 이중 주입 (Double Injection)

OpenClaw의 Spawn+Announce 패턴을 도입할 때 **동기/비동기 분리 조건을 누락**했습니다. delegate_task(동기)가 tool_result로 전체 데이터를 즉시 반환하는데, 다음 라운드에서 announce가 500자 요약을 다시 주입합니다. 서브에이전트 3개 위임 시 6건 메시지 중 50%가 중복입니다.

BEFORE

Round 3
경로1 **tool_result**
delegate_task 동기 반환 (전체 데이터)

↓ 같은 정보 재주입

Round 4
경로2 **announce**
system_event로 500자 요약 재주입

LLM이 같은 내용 2회 수신

AFTER

Round 3
경로1 **tool_result**
delegate_task 동기 반환 (전체 데이터)

↓ announce=False

Round 4
announce queue **비어있음**
0건 주입. 부모 polling은 유지

정보 1회만 수신. 50% 절감

3종 프론티어 비교

Claude Code
tool_result 단일 경로
announce 없음
이중 주입 없음

OpenClaw
agentTurn(동기): 콜백 직접
systemEvent(비동기): announce
동기 시 announce 미사용

autoresearch
파일 저장 (컨텍스트 차단)
L1-L3: 10K→10 토큰
이중 주입 없음

GEODE (After)
동기: tool_result만
비동기: announce 유지
OpenClaw 패턴 채택

동시성 제어

5	250ms	2
MAX_CONCURRENT	CoalescingQueue	depth limit

Trade-off: 동기/비동기 경로 분리

Claude Code처럼 announce를 완전히 제거하면 단순하지만 비동기 경로를 포기해야 합니다. OpenClaw 방식을 채택하여 **announce 기본값 True**를 유지하고, 동기 호출(delegate_task)에서만 **announce=False**로 비활성화합니다. 기존 비동기 경로(systemEvent)는 그대로 작동합니다.

변경 규모: 2파일 4줄. Open-Closed 원칙 준수

LLM Failover: 단일 Provider 의존 → 3-Provider Chain

Rate Limit·장애 시 파이프라인 전체 중단 방지 — CircuitBreaker 3-state + Exponential Backoff

단일 Provider에 의존하면 Rate Limit이나 장애 시 파이프라인 전체가 중단됩니다. Anthropic·OpenAI·ZhipuAI를 **Port/Adapter**로 추상화하고, Provider별 독립 **Circuit Breaker**를 배치했습니다.

3-PROVIDER FAILOVER CHAIN

Anthropic
claude-opus-4-6
↓ claude-sonnet-4-6
↓ claude-haiku-3-5

5 fails →

OpenAI
gpt-5.4
↓ gpt-5.3
↓ gpt-4o

5 fails →

ZhipuAI
GLM-5
최후 보루
80K context

CircuitBreaker 상태 머신 (Provider별 독립)

CLOSED
정상 운영

연속 5회 실패 →

OPEN
요청 차단

60초 후 →

HALF-OPEN
프로브 1회

에러 분류

Retryable

- Timeout
- RateLimit
- Connection

Exp. Backoff ±25% Jitter

Non-retryable

- 잔액 부족
- API Key 오류
- Context 초과

즉시 다음 Provider

핵심 설계 결정

메모리가 모델에 종속되지 않습니다. 4-Tier 메모리는 Provider와 분리되어 Failover 시에도 세션 학습 내용이 그대로 유지됩니다.

Port/Adapter 추상화로 새 Provider 추가 시 어댑터 1개만 구현. Thundering Herd 방지를 위해 ±25% Jitter 적용.

Context + Memory: 압축 전략 × 4-Tier 자동 학습

세션 종료 시 인사이트 휘발 방지 — PIPELINE_END → MEMORY.md 자동 축적 루프

서브에이전트는 부모와 독립된 200K 컨텍스트 윈도우를 갖기 때문에 **깊이 3이면 유효 컨텍스트가 600K로 선형 확장**됩니다. 임계값 초과 시 3-Phase 압축이 자동 발동합니다.

컨텍스트 압축 3-PHASE

80% WARNING	**Phase 1 SUMMARIZE** tool_result 블록 >5% 윈도우 → [summarized: N tokens]
95% CRITICAL	**Phase 2 ADAPTIVE PRUNE** 최신→과거 순 70% 예산 내 유지, first+last 2 보존
비상 폴백	**Phase 3 EMERGENCY PRUNE** prune_oldest_messages(keep_recent=10) 최후 수단

Karpathy P6 패턴: 4 chars/token 보수적 추정 → 실제보다 낮게 계산 = 안전 마진 확보

4-TIER MEMORY 피라미드

Session (Tier 3) — 최우선 TTL 기반 · 현재 작업 맥락	40%
Project (Tier 2) .geode/memory/PROJECT.md · max 200 lines	25%
Organization (Tier 1) 불변 가이드라인 · 공통 루브릭	25%
SOUL (Tier 0) — GEODE.md 에이전트 정체성 · Karpathy P7 패턴	10%

PIPELINE_END → 자동 학습 루프

분석 세션이 끝나면 Hook P85 **memory_write_back**이 인사이트를 .geode/memory/PROJECT.md에 자동 축적합니다. 최대 50개 인사이트 (oldest-drop rotation).

PromptAssembler: 6-Phase 동적 조립

수동 편집 → 변경 추적 불가 + 캐시 히트율 하락. SHA-256 해시와 ephemeral 캐시로 해결.

PROBLEM — 수동 편집의 두 가지 실패

① 변경 추적 불가: 어느 버전이 어떤 결과를 냈는지 재현 불가.
② 캐시 히트율 하락: 작은 수정 한 줄이 전체 해시를 바꿔 캐시 미스 유발.

SOLUTION — 6-Phase 조립 파이프라인

template (역할 정의) → override (노드별 오버라이드) → skill (키워드 매칭) → memory (MEMORY.md) → bootstrap (사용자 접두사) → SHA-256 (변경 추적)

Trade-off — 조립 복잡도 vs 캐시 안정성

각 Phase가 독립 청크로 분리되어 변경된 Phase만 해시가 달라집니다. cache_control 범위가 명확해져 캐시 히트율이 보존됩니다.

RESULT — cache_control: ephemeral

10%

2회차부터 입력 비용

7개 노드가 동일 시스템 프롬프트 공유 시 적용

SHA-256 변경 추적

조립 완료 시 전체 프롬프트를 SHA-256 해싱하여 버전 고정

동일 해시 = 동일 프롬프트 → 캐시 히트 보장

Phase 변경 시 HookSystem PROMPT_CHANGED 이벤트 발행

25 Skills 전체가 이 파이프라인을 통해 주입됨

MCP 단순화 + TokenTracker 자체 관측

Registry 병목 제거로 LLM 도구 탐색을 열고, 자체 비용 트레이싱으로 루프 비용을 제어합니다.

MCP 단순화 — Before → After

BEFORE: 3-Layer 병목

MCPRegistry (257줄) — LLM 탐색을 가로막는 중앙 레지스트리

Catalog — Registry에 종속, 독립 검색 불가

Proxy stub 9개 — 중복 래퍼, 유지 관리 부담

AFTER: 2-Layer 직관

MCPRegistry 삭제 → Catalog 검색 전용으로 축소

config.toml [mcp.servers]로 설정 통합

44개 서버 auto-discovery · Proxy stub 9개 제거

Trade-off — 제어 vs 탐색 자유도

Registry는 도구 목록을 완전 통제하지만 LLM의 런타임 탐색을 막습니다. Catalog로 전환하면 44개 서버를 동적 관리하면서도 HITL 4-Tier 권한 모델로 실행 경계를 유지합니다.

TokenTracker — 자체 비용 트레이싱

왜 만들었나

while(tool_use) 루프는 LLM 턴이 많습니다. LangSmith만으로는 모델별 비용 내역과 캐시 히트율을 추적하기 어려웠습니다.

ContextVar 싱글톤

모델, 입출력 토큰, cache_creation, cache_read 기록. 스레드 안전.

18개 모델 가격 레지스트리

Anthropic·OpenAI·GLM 실시간 비용 산출. 월별 JSONL 영속 저장.

/cost CLI 대시보드

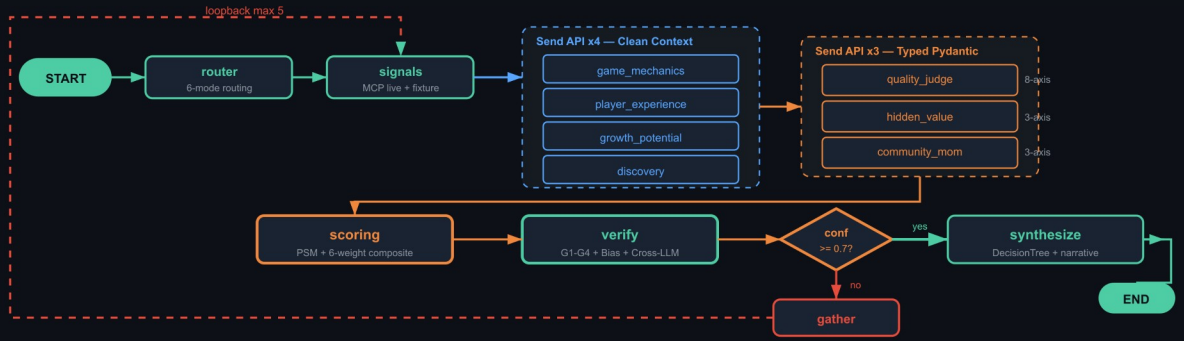
세션/일간/월간 비용 + 예산 소진율 실시간 표시

컨텍스트 **80%** 경고 → **95%** 위험 시 adaptive pruning 자동 발동

ProjectJournal: 프로젝트별 실행 이력 + 누적 비용 추적

Game IP Pipeline: 9-Node DAG + 5-Layer 검증

단일 LLM 평가의 편향을 병렬 전문가 4명 + 교차 검증 + PSM으로 극복합니다.



14-axis Rubric: Quality Judge (8) — gameplay, narrative, visual, audio, community, monetization, innovation, longevity
Hidden Value (3) — crossover, nostalgia, modding | Community Momentum (3) — sentiment, growth, advocacy

Scoring: Final = (0.25·PSM + 0.20·Quality + 0.18·Recovery + 0.12·Growth + 0.20·Momentum + 0.05·Dev) × (0.7 + 0.3·Conf/100)
Tier: S ≥ 80 | A ≥ 60 | B ≥ 40 | C < 40

5-Layer Verify: G1-G4 Guardrails → BiasBuster 6종 → Cross-LLM (Krippendorff α) → Confidence Gate ≥ 0.7 → Rights Risk

Design: Send API Clean Context (앵커링 방지) | Code Decision Tree (결정적 분류, LLM 아님) | Typed Pydantic (축별 강제) | DomainPort 교체 가능

Scoring Formula (adapter.py:91-98)

```
base = 0.25·PSM + 0.20·Quality + 0.18·Recovery
+ 0.12·Growth + 0.20·Momentum + 0.05·Dev
final = base × (0.7 + 0.3·conf/100)
```

Tier 기준

S ≥ 80 · A ≥ 60 · B ≥ 40 · C < 40

D-E-F 축 코드 결정 트리 (LLM 아님)

설계 결정 — Clean Context: 앵커링 편향 차단

"Analyst A의 S급 평가가 Analyst B를 편향시킵니다." 각 분석가에게 analyses: []로 전달 — 다른 분석가 점수 완전 차단. 7개 병렬 LLM 호출에서 독립 평가 보장.

14 rubric axes (8+3+3) · 6 cause classes · PSM Rosenbaum Gamma · conf gate loopback ≥0.7 · max 5 iterations

Fixture Results — Golden Set 검증

S 81.2 — **Berserk**
conf: 0.89 · bias: 0/6
cause: conversion_failure → marketing_boost

A 68.4 — **Cowboy Bebop**
conf: 0.85 · range: 63-75
cause: undermarketed → community_activation

B 51.7 — **Ghost in the Shell**
conf: 0.72 · range: 48-60
cause: discovery_failure → platform_expansion

5-Layer 검증 (N4)

L1 Guardrails — 스키마·범위·근거·일관성
L2 BiasBuster — 6종 편향 탐지 (arXiv:2403.00811)
L3 Cross-LLM — Claude×GPT α≥0.67
L4 Rights Risk · L5 Calibration 80/100

도메인 이식성: GEODE → REODE 실증

에이전트 코어 수정 0줄로 피봇. DomainPort Protocol이 도메인을 코어에서 분리합니다.

GEODE v0.30

Game IP 저평가 분석 에이전트

221 모듈 · 3,181 테스트

54 Tools + 44 MCP · 25 Skills

32 릴리스 무회귀

GameIPAdapter implements DomainPort

DomainPort Protocol (15 메서드, 6 카테고리)

Identity (3) · Analyst Config (2) · Evaluator Config (3)
Scoring (4) · Classification (5) · Fixtures (2)

```
_domain_ctx: ContextVar[DomainPort]
set_domain() / get_domain_or_none()
```

@runtime_checkable Protocol — 상속 없이 15개 메서드 구현하면 즉시 호환. Pipeline 노드는 get_domain_or_none()으로 조회 후 fallback.

DomainPort Protocol

2-Protocol 직교 설계

PipelineTemplate L1
실행 흐름 정의

LanguageAdapter L2
언어·포맷 변환

0줄
에이전트 코어 수정
core/graph.py · core/verification/ 무변경

↕
ContextVar 스왑으로
도메인 전환

REODE v0.21

Java 코드 마이그레이션 에이전트

209 모듈 · 3,274 테스트 · 34 Tools

3 Pipelines (Migration · Porting · Refactoring)

5-Gate Migration Scorecard

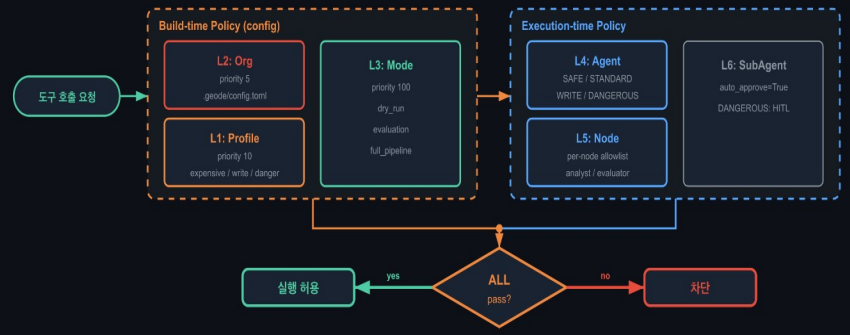
실증 수치 — Java 마이그레이션

241 소스 파일	**103K** Lines of Code
33 세션	**1,133** LLM 라운드

83/83 mvn compile+test 패스
5h 48m 자율 실행

Security + Gateway: Swiss Cheese 방어

Bash 3-Layer + Secret Redaction 8패턴 + ChannelBinding 세션 격리로 자율 실행 경계를 제어합니다.



Priority: L3(100) > L1(10) > L2(5) — 높은 숫자가 우선. 모든 레이어를 AND로 통과해야 실행 허용. dry_run은 모든 WRITE/DANGEROUS 차단.

Whitelist > Blacklist — allowed_tools 설정 시 denied_tools 무시. NodeScope(L5): analyst=memory_search, evaluator=steam_info. OpenClaw Policy Chain 패턴.

Layer 1 — 9 차단 패턴

`rm -rf /` · `sudo` · `>/etc/`

`curl|sh` · `wget|sh` · `mkfs.`

`dd if→/dev/` · `chmod 777 /`

`fork bomb :(){:|:&}`

Layer 2 — Resource Limits

CPU 30s 하드 캡

파일 크기 50MB 상한

자식 프로세스 64개 상한

preexec_fn setrlimit 적용

Layer 3 — HITL 4-Tier

SAFE — 30 prefix 자동 승인

WRITE — 개별 승인 필요

EXPENSIVE — 비용 일괄 승인

DANGEROUS — 순차 승인

Trade-off — safe-first 허용 시 파이프 주입 통과

cat file.txt | bash는 cat prefix로 SAFE 분류 → 파이프 주입이 통과됩니다. Layer 1의 curl/wget 차단이 원격 코드 실행을 방어합니다.

9 blocked patterns · CPU/FSIZE/NPROC limits · 8 secret patterns · 6-layer PolicyChain AND-logic · ChannelBinding 3 pollers · geode serve hitl=0

Secret Redaction — 8 패턴

`sk-ant-...` Anthropic API key

`sk-proj-...` / `sk-...` OpenAI

`ghp_...` / `gho_...` GitHub PAT/OAuth

`xoxb-...` / `xoxp-...` Slack

BashTool.to_tool_result() 경계에서 적용 — LLM 컨텍스트 진입 전 차단

Gateway — 채널 세션 격리

ChannelBinding

Slack · Discord · Telegram 3채널, 정적 바인딩 규칙으로 결정론적 라우팅

세션 격리

`gateway:{channel}:{id}[:{sender}][:{thread}]`
스레드별 독립 컨텍스트 — 동시 대화 격리

geode serve

hitl_level=0 완전 자율 · quiet=True · echo 2중 방어 (bot_message subtype + _last_ts)

PolicyChain 설계 원칙 — AND-logic

도구는 모든 정책을 통과해야 실행됩니다. Org 정책(priority=5)이 최우선. 6개 레이어 중 하나라도 거부하면 차단. Swiss Cheese: 단일 실패를 다음 레이어가 차단합니다.

Dual Trust Model

GEODE는 두 가지 신뢰 모델을 병행합니다. 자율 에이전트는 실행 전 입력을 차단(Input Gating)하고, 분석 파이프라인은 실행 후 출력을 검증(Output Validation)합니다.

CANNOT → Result: 제약이 품질을 보증한다

제약 없이 발산하는 LLM을 제어 가능한 결과로 수렴시키는 4가지 인과 경로

CANNOT

rm -rf / · sudo · curl|bash

bash_tool.py 9 regex + setrlimit — CPU 30s · FSIZE 50MB · NPROC 64

→

RESULT

보안 사고 0건

9개 패턴이 실행 차단, 리소스 초과는 커널이 강제 종료

CANNOT

다른 애널리스트 점수를 보고 평가

Clean Context — Send 상태에 analyses: [] 삽입, 각 애널리스트는 빈 배열만 수신

→

RESULT

앵커링 편향 구조적 차단

4 애널리스트 독립 병렬 → BiasBuster CV 검사로 2차 확인

CANNOT

API 키를 LLM 컨텍스트에 노출

redaction.py 8 regex — sk-ant-*, ghp_* 등 출력이 LLM에 도달하기 전 redact_secrets() 호출

→

RESULT

키 누출 0건 — [REDACTED] 치환

8종 패턴 전수 검열, LLM 컨텍스트 경계에서 강제 치환

CANNOT

신뢰도 미달 결과를 최종 출력

Confidence Gate ≥0.7 — 미달 시 gather 노드로 루프백, MAX 5회 반복 후 강제 진행

→

RESULT

자가 치유 루프 + 무한 루프 방지

MAX_ITERATIONS=5 하드캡 — weak_areas 기록으로 재실행 시 집중 영역 좁힘

설계 결정: 파이프라인 · 실행 · 보안

각 항목: [채택] vs [기각] — 왜 이 선택인가를 1줄로 정당화

파이프라인 · 실행

Plan-and-Execute DAG vs ReAct
결정론적 실행 순서 + 병렬 노드 — 재귀 추론 루프 없이 구조화된 다단계 가능

Send API 병렬 실행 vs 순차 처리
4 애널리스트 × 3 평가자 동시 — LangGraph Send가 상태 병합을 자동 처리

Clean Context (analyses: []) vs 전체 공유
앵커링 편향 구조적 차단 — 단일 설계 결정이 평가 품질에 가장 큰 영향

Confidence Gate ≥0.7 루프백 vs 단일 패스
수동 개입 없는 자가 치유 — MAX=5로 무한루프 차단

보안 · 검증

3중 보안 (regex+limits+redaction) vs 단일 샌드박스
Swiss Cheese — 각 레이어가 독립적으로 다른 공격 면을 차단

PolicyChain AND 체인 vs OR 허용
모든 정책 통과 필수 — Org 정책(priority=5)이 모든 하위 규칙 오버라이드

4-Tier HITL (SAFE/WRITE/DANGEROUS/EXPENSIVE) vs 이진
30개 안전 접두사 자동 승인 — 세분화로 마찰 최소화, 위험 제어

5-Layer Swiss Cheese 검증 vs 단일 종합 검사
G1-G4 구조 + BiasBuster 인지 + Cross-LLM 통계 + Rights 법적 + Calibration 실증

파이프라인 4 + 보안·검증 4 — 채택 근거는 코드 경로와 측정 결과로 뒷받침 (2/2는 다음 슬라이드)

설계 결정: 게이트웨이 · 인프라

결정론적·제로 레이턴시 라우팅부터 도메인 이식성까지 — 8가지 인프라 결정

게이트웨이 · 통신

정적 바인딩 라우팅 vs LLM 의도 라우팅
결정론적·제로 레이턴시·감사 가능 — LLM 라우팅은 비용·지연·불확실성 추가

Daemon Thread 폴링 vs asyncio 이벤트 루프
동기 LangGraph와 단순 통합 — 비동기 복잡도 없이 동시성 확보

Echo Prevention (subtype + _last_ts) vs DB dedup
경량·DB 의존 없음 — 3초 폴링 주기에 충분한 메모리 기반 dedup

계층형 세션 키 vs 플랫 UUID
gateway:ch:id:sender:thread — 스레드별 컨텍스트 격리, 사람이 읽을 수 있는 키

인프라 · 이식성

DomainPort Protocol + ContextVar DI vs ABC 상속
구조적 타이핑 — 상속 결합 없음, 스레드 안전 교체 가능

SqliteSaver + MemorySaver fallback vs 항상 인메모리
프로덕션 크래시 복구 + 개발 경량 — 단일 팩토리로 환경별 전환

서버 사이드 점수 계산 vs LLM composite_score
(axes_sum-8)/32×100 — 투명·검증 가능, LLM 수치는 비교 참고만

Serve = REPL 팩토리 + hitl_level=0 vs 별도 코드베이스
_build_agentic_stack() 재사용 — 동작 플래그만 다름, 코드 중복 0

이식성 증명: GEODE → REODE
동일 6-Layer 코어 — 도메인 플러그인만 교체, core/graph.py 변경 0줄

SUMMARY — GEODE v0.30 + REODE v0.21

Harness Engineering — 전체 지표

확률적 시스템을 수렴시키는 4개 축의 실증 결과 (GEODE + REODE 합산)

컨텍스트 제어
PromptAssembler 6-Phase · SHA-256 캐시 90% 절감

실행 루프
AgenticLoop while(tool_use) · 54 tools × 50 rounds

검증
Swiss Cheese 5-Layer · 신뢰도 69.4 → 99.8%

관측
36-Event Hook Observer · CUSUM 드리프트

GEODE v0.30 — 코드베이스

184	48K+	3,159	25
Python 모듈	Lines of Code	테스트 함수	semver 릴리스

파이프라인 · 도구
9-Node StateGraph DAG
54 Tools · 44 MCP 서버 · 25 Skills
3채널 Gateway (Slack/Discord/Telegram)

보안 · 검증
9 Bash 차단 · 8 Secret 탐지
6-Layer PolicyChain · 4-Tier HITL
Calibration 80/100 통과

REODE v0.21 — 이식성 증명

209	3,274
모듈	테스트

103K LoC 마이그레이션 · 83/83 통과
33 세션 / 1,133 라운드 / 5h 48m

core 코드 변경 0줄
Python ≥3.12 · LangGraph · Anthropic SDK · FastAPI · Redis · SQLite · mypy strict

GEODE(184) + REODE(209) = 393 모듈 · 6,433 테스트 — 확률론적 시스템을 루프와 가치로 수렴

확률적 시스템(LLM)은 제어 없이 발산합니다.

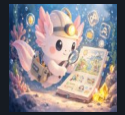
발산을 루프와 가치로 수렴시키는 하네스를 만듭니다.

Harness Engineering은 기법이 아니라 규율(discipline)입니다 — 제약, 루프, 검증을 체계적으로 설계하는 방법론.

Triple Loop — 자기 증강 하네스

META 설계 루프 → DEV 개발 루프 → RUNTIME 실행 루프 → HOOKS 관측 루프 ↺

META 루프가 DEV를 설계하고, RUNTIME이 실행하며, HOOKS가 관측하여 META로 피드백



Contact
github.com/mangowhoiscloud
mangowhoiscloud@gmail.com

Harness Engineering 3가지 가치 명제

① Scaffold Design — 제약으로 품질 보증

CANNOT 규칙을 코드로 집행 — 9 Bash 차단 · 8 Secret 탐지 · 6-Layer PolicyChain · 36 Event Hooks

결과: 보안 사고 0건 · 키 누출 0건 · 범위 외 호출 0건

② System Architecture — 6-Layer + DomainPort

L0 CLI → L5 Domain Plugins. DomainPort Protocol(15 methods) ContextVar DI.

증명: GEODE → REODE — core/graph.py 변경 0줄

③ Domain Reasoning — PSM + 5-Layer + CUSUM

Swiss Cheese: G1-G4 · BiasBuster · Cross-LLM α≥0.67 · CUSUM 드리프트 관측

결과: Calibration 80/100 · Berserk S-Tier(80-86) 재현 가능

GEODE EVOLUTION

Timeline: 도메인 파이프라인 → 범용 자율 실행 하네스

과제 전형용 도메인 파이프라인에서 범용 자율 실행 하네스로. 35일 피봇.

v0.1 — v0.7 Jan – Feb 2026

Phase 1 — 과제 합격

Game IP 가치추론 DAG. 넥슨 네비게이터실 과제 전형.

14축 루브릭 + PSM 인과추론 + LangGraph StateGraph.

`LangGraph StateGraph`
도메인 종속 파이프라인 시대

v0.8 PIVOT Mar 초 2026

Phase 2 — Autonomous Pivot

면접 탈락 → 자율 피봇. 도메인 종속 파이프라인에서 범용 자율 실행 하네스로 전환.

DomainPort Protocol 설계로 도메인 교체 가능성 확보.

`DomainPort Protocol`
범용 하네스 설계 전환점

v0.24 — v0.30 Mar 중후 2026

Phase 3 — Harness 완성

3중 구조 완성: Scaffold → Runtime → Loop.

Scaffold: CLAUDE.md + CI 체계
Runtime: 45 Hooks + 21 Skills
Loop: while(tool_use) 자율 실행

`while tool_use: loop()`
범용 자율 실행 하네스 완성

FORK: REODE Mar – May 2026

Fork — DomainPort 실증

Java 8→22, Spring 4.3→6.1. 241소스 103K LoC 레거시 마이그레이션.

83/83 빌드 통과. 에이전트 코어 수정 0줄.

`DomainPort: 0 core changes`
하네스 범용성 검증 완료

과제 전형용 도메인 파이프라인에서 범용 자율 실행 하네스로. 35일 피봇.